

# CS 485/585 – Fall 2026

## Lab 1: TCP Chat Room – Sockets Programming Assignment

Version: v1

Handed Out: Thursday, September 10, 2026

**Due: Thursday, October 1, 2026**

### Objectives

- Apply socket programming in C using the BSD sockets API.
- Implement a plain-text application-layer protocol from a written specification.
- Parse custom text commands from raw byte input.
- Handle TCP's byte-stream delivery model by implementing your own message framing.
- Apply mutex-based synchronization to protect data shared across threads.

### Introduction

In this assignment, you will create a chat room on a single computer where you and your (imaginary) friends can chat with each other. The following steps will be required to achieve this:

1. Create a server program that runs on a specific port passed via the command line.
2. Create a client program that can join this server.
3. The client needs a display name and a password to enter the chat room. (Assume all clients use the same password but a different display name.) The password is restricted to a maximum of 5 alphanumeric characters. Anything over 5 characters can be treated as invalid. The display name is a maximum of 8 characters long.
4. The job of the server is to accept connections from clients, get their display name and password (in plaintext), verify that the password is correct, and then allow clients into the chat room.
5. When any client sends a message, the display name is shown before the message, followed by a colon (:), and the message is delivered to all other current clients.
6. Clients can type any text message, or can type one of the following shortcut codes to display specific text:
  - (a) Type `:)` to display `[feeling happy]`
  - (b) Type `:(` to display `[feeling sad]`
  - (c) Type `:mytime` to display the current time
  - (d) Type `:+1hr` to display the current time + 1 hour  
Note: for `:mytime` and `:+1hr`, format time as "Weekday Month Day Time Year". For example: `Mon Aug 13 08:23:14 2012`.
  - (e) Type `:Users` to display a list of all active users.

- (f) Type `:Msg <username> <message>` to send a private message to a specific user.
- (g) Type `:Exit` to close your connection and terminate the client.
- (h) Fun part (not graded): `\` overrides the next word until a space or newline. For example, `\:mytime` will print `:mytime` instead of the actual time.

## What will you learn?

- Basic socket programming in C to create a client-server application.
- How multiple clients connect to a single server, and how to manage several simultaneous connections.
- How to keep multiple persistent TCP connections alive.
- How to use POSIX threads (pthreads) to handle clients concurrently, and how to protect data that's shared across those threads.
- How TCP's byte-stream model means your application, not the OS, is responsible for figuring out where one message ends and the next begins.
- Text parsing in C to develop a custom application-layer protocol.

## Which programming language to use?

You are required to use **C**. Your code will be tested on the CS lab machines by the instructional team.

Your submission must include a **Makefile** that builds both `server` and `client` binaries using the following flags: `-std=gnu11 -Wall -Wextra -pthread -O0 -g`. Code that does not compile will not be graded. Compiler warnings will not cause a build failure on their own (no `-Werror`), but pay attention to them.

**Resources:** see the manual pages for `pthread_create(3)`, `pthread_mutex_lock(3)`, and `ctime(3)`.

## Concurrency and Memory Management Requirements

Your server must handle multiple clients **concurrently** using pthreads: spawn one thread per connected client to handle that client's messages.

Because multiple threads will read and modify the list of currently connected clients (adding a client on join, removing one on `:Exit`, and iterating the list to broadcast messages), that shared list must be protected with a `pthread_mutex_t`. Any thread accessing the list, including while broadcasting, must hold the lock for the full duration of that access. Your server will be tested for correct behavior under concurrent client activity.

### Required implementation

The client list must be implemented as a fixed-size array of exactly `MAX_CLIENTS = 10` slots, each tracking whether it is currently in use. This baseline implementation avoids dynamic allocation. If a client attempts to connect while the server is already at capacity, the server must reject the

new connection without adding it to the active client list. The rejected client should receive the message: **“Server full”**.

### Extra credit

Implement the client list using dynamic memory allocation (e.g., a linked list, with `malloc/free` per client) instead of a fixed array. In this case, your locking must also prevent one thread from freeing a client’s memory while another thread is still reading it. The `MAX_CLIENTS = 10` still holds.

## Instructions/Expected Outputs

The autograder expects a very specific output from your programs. Please make sure you follow all conventions we set and don’t add extra spaces/newlines to your output. As a rule of thumb, there should be no empty lines in your program’s output, and spaces should be added after a username (e.g., `<username>: message` instead of `<username>:message`).

**Note: Neither program may print anything on stdout beyond what this specification calls for. This applies to your server as well as your client. For debugging you should instead use `fprintf(stderr, ...)`, which the autograder will ignore.**

### Connection Establishment and Password Checking — Single Client

*Note: the password is restricted to a maximum of 5 characters.*

You will create two programs: a client and a server. Each program takes the following command-line parameters: the client takes the server’s IP address and listening port, the username, and the password (all clients use the same password); the server takes its listening port and the password. Use `getopt_long()` for argument parsing, with the following flags:

Command: `./server --port <port> --password <password>`

Output: `Server started on port <port>. Accepting connections`

Command: `./client --host <host> --port <port> --username <username> --password <password>`

Output (on new Client): `Connected to <host> on port <port>`

Output (on Server): `<username> joined the chatroom`

If the password is incorrect, the client should receive a failure message: `Incorrect password`.

### Connection Establishment and Password Checking — Multiple Clients

The server must handle multiple clients connecting to it. Running the client command again (with a different username) should behave the same way, and the server should inform already-connected clients that a new client has joined.

Command: `./client --host <host> --port <port> --username <username2> --password <password>`

Output (on new Client): `Connected to <host> on port <port>`

Output (on Server): `<username2> joined the chatroom`

Output (on all other Clients): `<username2> joined the chatroom`

You don’t have to check for unique usernames; test cases will only supply unique usernames.

A connected client maintains a persistent TCP connection with the server that's only terminated when the user types `:Exit`. Do not assume a client will be forcibly terminated; a client is removed only when it executes `:Exit`.

## Server at Capacity

If the server is already at `MAX_CLIENTS = 10` connected clients, it must check capacity **before** reading the new connection's username or password, and reject the connection immediately.

```
Command: ./client --host <host> --port <port> --username <username> --password <password>
Output (on rejected Client): Server full
```

The server prints nothing when rejecting a connection at capacity, matching the behavior for an incorrect password. Once a connected client disconnects (via `:Exit`), a subsequent client should be able to join normally, provided the server is no longer at capacity.

## Chat Functionality

After connecting, clients can type messages that get sent to the server when the user presses Enter. All text before the newline should be sent to the server, displayed on the server's screen, and broadcast to all other clients.

```
Command (on a connected client, username <username>): Hello Room
Output (on Server): <username>: Hello Room
Output (on all other Clients): <username>: Hello Room
```

You don't have to consider messages longer than 100 characters; no need to handle or test inputs beyond that length.

## Chat Shortcuts

Emotion shortcuts `:)` and `:(` are broadcast to the server and all other clients, **excluding** the original sender.

```
Command (on a connected client): :)
Output (on Server): <username>: [feeling happy]
Output (on all other Clients): <username>: [feeling happy]
```

The time shortcuts `:mytime` and `:+1hr` are broadcast to **all** connected clients, **including** the original sender.

```
Command (on a connected client): :mytime
Output (on Server, and all Clients including sender): <username>: Wed Sep 24 12:31:44 2025
```

## Additional Chat Commands

`:Users` lists all actively connected clients as a comma-separated string with the prefix `Active Users:`.

```
Command (on a connected client, <username>): :Users
Output (on the same client): Active Users: <username1>, <username2>, <username3>
Output (on Server): <username>: searched up active users
```

:Msg <username2> <message> sends a private message to a specific user. You will not be tested against sending messages to a non-existent user.

Command (on a connected client, <username>): :Msg <username2> CS485/585 is awesome

Output (on <username2>): [Message from <username>]: CS485/585 is awesome

Output (on Server): <username>: sent message to <username2>

## Leaving the Chatroom

Command (on a connected client): :Exit

Output (on Server): <username> left the chatroom

Output (on all other Clients): <username> left the chatroom

## Grading Scheme (100 pts total)

- Single server-client program sets up connection: 20 points
- Single server, multiple clients able to connect concurrently: 20 points
- Server receives from any client, broadcasts to all (thread-safe): 20 points
- Login and password implementation: 5 points
- Text parsing for shortcut codes: 20 points
- Correct mutex usage / no data races on the shared client list, including correctly rejecting connections at MAX\_CLIENTS capacity: 10 points
- `collaboration-reflection.md` completed and well-articulated: 5 points
- Compilation errors are not acceptable.
- **Extra credit:** dynamic (malloc/free-based) client list with correct thread-safe memory management: up to 5 points

*Note: the scores above assume satisfactory performance on the Lab Quiz (see below).*

## Lab Quiz

After the Lab 1 deadline, you will complete a short in-class quiz about the code you submitted. This quiz does not add points to your Lab 1 score. Instead, it verifies that your submission reflects your own understanding of the code you wrote. If you are unable to adequately explain or demonstrate understanding of your submitted code, your Lab 1 grade may be reduced by up to 50%. In cases where a student cannot demonstrate any meaningful understanding of their own submission, a grade of zero may be assigned for the lab. This quiz will take place on **Tuesday, October 6, 2026**, during class.

## Programming Do's and Don'ts

- Use `getopt_long()` for command-line argument parsing rather than parsing `argv` by hand.
- Your server should bind to `127.0.0.1` as the host. You will only be using IPv4 to bind to localhost; do not attempt to bind to an IPv6 address.

- TCP delivers a stream of bytes, not discrete messages. A single `recv()` call may return a partial line, multiple lines at once, or nothing new. Do not assume that one `recv()` call corresponds to one client command.
- Protect the shared list of connected clients with a mutex. Any thread that reads or modifies the list, including while broadcasting a message, must hold the lock for the entire duration of that access.
- When passing a client's slot index (or connection info) into a new thread via `pthread_create()`, do not pass the address of a loop variable directly. If that variable gets reused on the next loop iteration before the new thread reads it, two threads can end up believing they own the same client. Give each thread its own dedicated memory for this value instead (e.g., a small heap-allocated struct passed as the thread argument).
- Build each line of server-side output fully (e.g., with `snprintf()` into a buffer) before printing it in a single call. Multiple client threads may be writing to the server's stdout at the same time; printing a line across more than one call risks another thread's output landing in the middle of it.
- It is recommended to implement `:Exit` functionality first. The autograder relies on this command to cleanly terminate client connections after each test, and your thread cleanup logic depends on it too. Failure to implement this correctly will likely cause most, if not all, subsequent test cases to fail.
- Set `SO_REUSEADDR` on your listening socket before you `bind()`. Without it, restarting your server on the same port shortly after shutting it down fails with "Address already in use", because the old socket is still lingering in `TIME_WAIT`.
- C buffers stdout differently depending on where it points: line-buffered to a terminal, fully buffered to a pipe or file. The autograder runs your programs under `stdbuf` so this will not affect your grade. If you ever redirect your output and it seems to vanish, that is why. Using `fflush(stdout)` or `setvbuf(stdout, NULL, _IOLBF, 0)` should fix those problems.

## AI and Collaboration Policy

Consistent with class policy, collaboration with classmates and LLMs is permitted and encouraged, provided it enhances rather than replaces your learning experience. Refer to these metrics to determine if your collaboration is healthy and permitted:

- Do not have the assignment open at the same time as you engage with the LLM.
- Do not copy and paste code, including asking an LLM to "add thread synchronization" or "fix my mutex" directly on your own file.
- Write and test your files iteratively, break up the work into smaller tasks and address them one at a time.

## What to Submit

Please implement all functionality described above in `server.c` and `client.c`, along with a `Makefile` that builds both programs.

You will be given a private GitHub repository for this assignment, with your instructor as the owner and your TA and yourself as collaborators. All of your code must be committed and pushed to this repository by the deadline. The repository includes the autograder as a submodule, so you can see how your code will be tested. Every commit you push will automatically trigger the autograder via GitHub Actions.

You must also fill out the `collaboration-reflection.md` document to detail the nature of your collaboration with classmates and LLMs.

## Further Questions?

Please use the Piazza thread **Lab 1 Help Thread** for posting all doubts and questions related to Lab 1.